TRIBHUVAN UNIVERSITY

INSTITUTE OF SCIENCE AND TECHNOLOGY

HIMALAYA COLLEGE OF ENGINEERING

CHYASAL,LALITPUR

Lab Report No:- 03

Title:- IMPLEMENTATION OF 16-Round key generation and S-Box function in DES.

Submitted By:-

Name :- Anil Pal

Roll No:- 5

Submitted To:-

Department Of CSIT

Date Of Submission:- 2082/12/11

CamScanner

LAB3: IMPLEMENTATION OF 16-Round key Generation and S-Box function in DES.

Objective:

To implement key components of the Data Encryption standard (DES) using C programming:

1. Generate 16 round key from a 64 bit key.
2. Implement the S-box substitution function to convert a 48-bit input into a 32-bit output.

Theory:

The Data encryption standard is a symmetric block cipher that encrypts 64-bit blocks using a 56-bit key over 16 rounds. DES uses substitution & permutation operation to provide confusion & diffusion in encrypted data:

1. 16 round key generation

- The original 64-bit key is reduced to 56 bits using parity drop (PC-1).
- The 56 bit key is split into two 28-bit halves (C & D).
- Both halves are shifted left according to a shift schedule for each round.
- The halves are then combined & passed through compression permutation (PC-2) to generate a 48-bit round key.
- This process produces 16 unique round keys, one for each DES round.

2. S-Box substitution

- The 48-bit input to the S-Boxes is divided into 8 6-bit blocks.
- For each block:
  - The first & last bit determines the row of the S-Box.
  - The middle 4 bit determine the ~~row~~ column of S-box.
  - The value from the S-box table is convert to 4-bit binary output.
  - Combining outputs from all 8 S-boxes produces a 32-bit substituted output, introducing non-linearity & enhancing security.

CamScanner

Q.1) Program to generate 16 round key in DES

```
#include<stdio.h>
#include<string.h>
unsigned char key[8]={0x13,0x34,0x56,0x78,0x98,0xAB,0xCD,0xEF};
unsigned char round keys[16][6];
unsigned char pc1[56]={
  57,49,41,33,25,17,9,1,58,50,42,34,26,18,
  10,2,59,51,43,35,27,19,11,3,60,52,44,36,
  63,55,47,39,31,23,15,7,62,54,46,38,30,22,
  14,6,61,53,45,37,29,21,13,5,28,20,12,4};
unsigned char pc2[48]={
 14,17,11,24,1,5,3,28,15,6,21,10,
 23,19,12,4,26,8,16,7,27,20,13,2,
 41,52,31,37,47,55,30,40,51,45,33,48,
 44,49,39,56,34,53,46,42,50,36,29,32};

unsigned char SHIFT-SCHEDULE[16]=
  {1,1,2,2,2,2,1,2,2,2,2,2,1,2,2,1};

unsigned char leftShift(unsigned char value, int shifts)
{
 return ((value<<shifts)|(value>>(8-shifts))) & 0xFF;
}
Void permute(unsigned char *input, unsigned char *output,
      unsigned char * permutation, int length)
{
 for(int i=0; i<length; i++)
  {
   int bit = permutation[i]-1;
   output[i/8]=((input[bit/8]>>(7-(bit%8))) & 1)<<(7-(i%8));
  }
}
void printBinary(unsigned char *data, int bytes)
{ for(int i=0; i<bytes; i++)
  {for(int j=7; j>=0; j--)
   {
    printf("%d",(data[i]>>j) & 1);
   }
   printf(" ");
  }
}
```

```
void generate Round keys ()
{
unsigned char permuted key [7]={0};
Permute (key, permuted key, pc1, 56);
Printf("\n key After pc1 permutation:\n");
Print Binary( permuted key, 7);
Printf("\n\n");
unsigned char left[4]={0}, right[4]={0};

memcpy (left, permuted key, 4);
memcpy (right, permuted key+4, 3);

printf("initial Left half:\n");
print Binary (left, 4);
Printf("\n");

Printf("Initial right half:\n");
Print Binary( right, 4);
Printf("\n\n");
for (int round =0; round <16; round++)
{ left[0]= leftShift(left[0], SHIFT_SCHEDULE [round]);
  right[0]= left shift (right[0], SHIFT_SCHEDULE [round]);

 unsigned char combined [8]={0};
memcpy( combined , left, 4);
memcpy( combined +4, right, 4);

 permute (combined, round keys [round], pc2, 48);
}
}

int main()
{
 printf(" original key (in Hex):");
  for (int i=0; i<8; i++)
    printf("%02X", key[i]);

    printf("\n Original key (in Binary):\n");

    print Binary (key, 8);
```

```
printf("\n");
generateRoundKeys();
printf("\n--- Round keys---\n");
  for(int i=0; i<16; i++)
  {
    printf("Round %.2d key:", i+1);
    printBinary(roundKeys[i], 6);
    printf("\n");
  }
  printf("Name: Anil Pal  Roll no:5");
  return 0;
}
```

```
C:\Users\Dell\OneDrive\Desk1

Original Key (in Hex): 13 34 56 78 90 AB CD EF
Original Key (in Binary):
00010011 00110100 01010110 01111000 10010000 10101011 11001101 11101111

Key After PC1 Permutation:
11110000 11001100 10101010 00011010 01011100 01101110 10001111

Initial Left Half:
11110000 11001100 10101010 00011010
Initial Right Half:
01011100 01101110 10001111 00000000


---- Round Keys ----
Round  1 Key : 11001011 00111100 01000011 00111100 01100101 01110110
Round  2 Key : 11001001 00111100 01010011 00101101 01000101 11110110
Round  3 Key : 11000101 01111100 01010010 00101101 01100101 11110010
Round  4 Key : 11000111 01111101 00000010 00101101 01000111 01110110
Round  5 Key : 11001011 00111101 00000011 00111100 01000101 11110110
Round  6 Key : 11001001 00111100 01010011 00101101 01000101 11110110
Round  7 Key : 11001001 01111100 01010010 00101100 01100111 11110010
Round  8 Key : 11000101 01111101 00010010 00111101 01100111 01110010
Round  9 Key : 11000111 00111101 00000011 00111100 01000111 01110010
Round 10 Key : 11001011 00111100 01000011 00111100 01100101 01110110
Round 11 Key : 11001001 01111100 01010010 00101100 01100111 11110010
Round 12 Key : 11000101 01111101 00010010 00111101 01100111 01110010
Round 13 Key : 11000111 01111101 00000010 00101101 01000111 01110110
Round 14 Key : 11001011 00111101 00000011 00111100 01000101 11110110
Round 15 Key : 11001001 00111100 01010011 00101101 01000101 11110110
Round 16 Key : 11001001 01111100 01010010 00101100 01100111 11110010
Name:Anil Pal Roll no:5
Process returned 0 (0x0)   execution time : 0.228 s
Press any key to continue.
```

CamScanner

Q.2.) Program to implement function of S-box in DES.

```
#include <stdio.h>
# include <string.h>
int S-BOX [4][16]=
{
 {14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},
 {0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},
 {4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},
 {15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}
};
int binary To Decimal (const char *binary, int length)
{int value=0;
 while (length--)
 { value= value*2 +(*binary++-'0');
  }
 return value;
 }
void decimal to Binary (int number, char *binary)
 { for(int i=3; i>=0;i--)
  { binary [3-i]= (number & (1<<i))?'1':'0';
   }
  binary [4]='\0';
  }
int main()
{ char input[49];
 char final output[33]=" ";
 printf("Enter a 48-bit binary input:");
 Scanf("%48s", input);
 if(strlen(input) != 48)
 { printf ("Error: input must be exactly 48 bits! \n");
   return 1;
   }
 printf("\n S-Box Results: \n");
```

```
for (int i=0; i<8; i++)
{ int row=(input[i*6]-'0')*2+(input[i*6+5]-'0');
  int col = binaryToDecimal(input + i*6 + 1, 4);
  int output = S_Box[row][col];

  char binaryoutput[5];
  decimalToBinary(output, binaryoutput);

printf("S-Box %d : Decimal = %d, Binary = %s\n", i+1, output, binaryoutput);

strcat(finaloutput, binaryoutput);
}

printf("\n ~~Enter~~ Final Combined 32-bit output: %s\n", finaloutput);

printf("\n Name: Anil Pal  Roll no:5");

return 0;
}
```

```
C:\Users\Dell\OneDrive\Deskt  ×  +  ⌄

Enter a 48-bit binary input: 101110101100101101100110101011010011100101001111

S-Box Results:
S-Box 1: Decimal = 11, Binary = 1011
S-Box 2: Decimal = 2, Binary = 0010
S-Box 3: Decimal = 1, Binary = 0001
S-Box 4: Decimal = 8, Binary = 1000
S-Box 5: Decimal = 9, Binary = 1001
S-Box 6: Decimal = 6, Binary = 0110
S-Box 7: Decimal = 8, Binary = 1000
S-Box 8: Decimal = 1, Binary = 0001

Final Combined 32-bit Output: 10110010000110001001011010000001

Name: Anil Pal  Roll no:5
Process returned 0 (0x0)   execution time : 21.563 s
Press any key to continue.
```

DISCUSSION

In this experiment, a program to generate 16 round key from a given 64 bit key, The program applied permutation, splitting shifting & compression operations for each round to produce 16 48-bit round keys. And also, discussed about S-box generation. in S-box Each block was substituted according to the S-box table to produce 4 bit outputs.

CONCLUSION:

In the last, we were able to implement 16 round key generation & S-Box function implementation using C program.

CamScanner